CSCE 355: Turing Machine Assignment 2

Overview

In this assignment you will build five deterministic single-tape Turing machines in JFLAP, one per problem below. Each machine decides (accepts or rejects) a language – your machine does not need to leave any particular output on the tape, it only needs to halt in an accepting state on strings that belong to the language, and halt in a non-accepting state (or fail to halt) on strings that do not.

You will submit five `.jff` files. Each will be tested automatically against 20 provided test strings (10 that your machine must accept, 10 that it must reject), and each problem is worth 5 points, for 25 points total.

Getting JFLAP

This assignment uses JFLAP, a free tool for building and simulating automata. Download it directly from the official site:

`https://www.jflap.org`

JFLAP is a `.jar` file and requires a working Java installation (Java 8 or later is recommended). It is not included in this assignment package – download it yourself from the link above. Do not use a third-party mirror.

Determinism Requirement

Every machine you submit must be deterministic. If a machine is found to be nondeterministic, it receives **zero credit** for that problem, regardless of whether it produces correct accept/reject results on the test strings. This is checked automatically by the grading script before any test strings are even run.

A Turing machine is deterministic if, for every state and every tape symbol, there is *at most one* transition that applies. Concretely, watch out for these two situations:

(a) **Two transitions leaving the same state on the same read symbol.** If state q_i has two different outgoing transitions that both read the symbol `0` (even if they write different symbols, move in different directions, or go to different states), the machine is nondeterministic at q_i.

(b) **A transition that can apply simultaneously with another.** In a standard JFLAP Turing machine, every transition reads some tape symbol (including the blank symbol, shown as $\square$), so this usually reduces to case (a) above – but double check any state with several outgoing arrows to make sure no two of them share a read symbol.

How to check for determinism in JFLAP

Method 1: Test → Highlight Nondeterminism. With your machine open in the editor, use the menu option **Test → Highlight Nondeterminism**. JFLAP will open a new view of your machine with any nondeterministic states highlighted. If nothing is highlighted, your machine passes this check. If any state is highlighted, inspect its outgoing transitions and remove or fix the conflicting one(s).

Method 2: Watch the number of simultaneous configurations while stepping. A deterministic machine only ever has *one* active configuration at any point during simulation. When you step through your machine (**Input → Step**), if you ever see more than one configuration displayed at the same time – or if JFLAP asks you to freeze/remove configurations because there are too many active at once to display – that is a clear sign your machine has taken a nondeterministic branch somewhere. A correctly built deterministic machine for this assignment should never show more than one configuration box during stepping.

Fix any state flagged by either method before submitting. The autograder performs an equivalent check on the raw transition table of your `.jff` file, so a machine that fails determinism in JFLAP will also fail it (and receive 0 points for that problem) when graded.

Problems

Each problem below is worth 5 points. Alphabet symbols other than those listed (and the blank symbol) should not appear in valid input, and your machine does not need to handle them specially.

1. **Palindrome.** Input alphabet $\{0, 1\}$. Accept if and only if the input string is a palindrome (reads the same forwards and backwards). The empty string counts as a palindrome.
   Files: `problem1_palindrome.jff`, `problem1_palindrome_tests.txt`

2. **Unary Addition Check.** Input alphabet $\{0, \#, =\}$. Input strings are of the form $0^m\#0^n = 0^p$ (unary numbers m, n, p separated by `#` and `=`). Accept if and only if $m + n = p$.
   Files: `problem2_addition.jff`, `problem2_addition_tests.txt`

3. **Binary Increment Check.** Input alphabet $\{0, 1, \#\}$. Input strings are of the form $x\#y$ where x and y are binary numbers. Accept if and only if y equals $x + 1$.
   Files: `problem3_increment.jff`, `problem3_increment_tests.txt`

4. **Copy Check.** Input alphabet $\{0, 1, \#\}$. Input strings are of the form $w\#w'$ where $w, w' \in \{0, 1\}^*$. Accept if and only if $w = w'$ (the two halves are identical).
   Files: `problem4_copy.jff`, `problem4_copy_tests.txt`

5. **$0^n1^n2^n$.** Input alphabet $\{0, 1, 2\}$. Accept if and only if the input has the form $0^n1^n2^n$ for some $n \geq 0$ (some number of 0's, followed by the same number of 1's, followed by the same number of 2's).
   Files: `problem5_anbncn.jff`, `problem5_anbncn_tests.txt`

Test String Files

Each `*_tests.txt` file contains exactly 20 lines:

- **Lines 1–10** are strings your machine must **accept**.

- **Lines 11–20** are strings your machine must **reject**.

A blank line represents the empty string (λ). You can load a test file directly into JFLAP's batch simulator via **Input** → **Multiple Run**, then **File** → **Open** the `*_tests.txt` file from within that window to load all 20 strings at once, and click **Run Inputs** to see accept/reject results for each row.

Do not edit the `*_tests.txt` files. They will be replaced with the original versions before grading, so any changes you make will simply be discarded – editing them will only make it harder for you to debug locally with an inaccurate picture of what will actually be graded.

Testing Your Machines

You have two complementary ways to test your work as you go:

1. **Inside JFLAP (interactive).** Use **Input** → **Step** to walk through individual strings one transition at a time and watch the tape, or **Input** → **Fast Run** for a quick single-string accept/reject check, or **Input** → **Multiple Run** loaded with the provided `*_tests.txt` file to check all 20 strings for a problem at once.

2. **From the command line (automated).** This assignment folder includes `test_tm.py`, the same grading script that will be used to grade your submission. From inside this folder, run:

   ```
   python3 test_tm.py
   ```

   This will parse all five of your `.jff` files, check each for determinism, run all 20 test strings per problem, and print a full report along with your total score out of 25. Run this as often as you like while working – it is exactly what will be run when your assignment is graded (with the test files and this script itself refreshed to their original versions first, so make sure you are testing against the originals).

Grading

Each of the five problems is worth 5 points, for 25 points total. For a given problem, you receive full credit (5 points) only if *both* of the following hold:

- Your submitted machine is deterministic (see above), **and**
- Your machine produces the correct accept/reject result on all 20 test strings for that problem.

Otherwise, that problem receives 0 points. There is no partial credit within a single problem.

Submission Instructions

1. Open **YOURUSERNAME.txt** and put your university username as the *first line* of the file (and only your username – nothing else on that line). This is how the grading script identifies your submission.

2. Build and save your five Turing machines into the provided `.jff` files (`problem1_palindrome.jff` through `problem5_anbncn.jff`), overwriting the empty templates.

3. Verify all five machines are deterministic and pass their test strings using `python3 test_tm.py` as described above.

4. Submit this entire folder (or a zip of it) as instructed by your TA/instructor. Do not rename any of the provided files.